

Consumer Electronics

Complaint Resolution System

End-to-End Workflow Documentation

Architecture · Microservices · LLM Models · Data Flow · Cross-References

v 2.0.0 · Samsung Consumer Electronics · FastAPI + Next.js + OpenAI

13

Python Modules

25+

API Endpoints

3

LLM Models

8

Samsung Products

6

Decision Codes

4

Core Data Files

Table of Contents

- 1 System Architecture Overview
- 2 Tech Stack
- 3 Microservices / Logical Modules
- 4 LLM Models Used
- 5 Data Files & Schemas
- 6 End-to-End Workflow — Step by Step
 - 6.1 Email Ingestion & Classification
 - 6.2 FAQ Branch
 - 6.3 Complaint Branch — AI Processing Pipeline
 - 6.3.1 Step 0 — Pre-check: Customer Guard
 - 6.3.2 Step 1 — AI Extraction
 - 6.3.3 Step 2 — Validation Engine
 - 6.3.4 Step 3 — Decision Engine & Customer Grounding
 - 6.3.5 Step 4 — RAG Injection (Conditional)
 - 6.3.6 Step 5 — Auto Email Response
 - 6.3.7 Step 6 — Save to Dashboard
- 7 Cross-Reference Map
- 8 Frontend (Next.js) Layer
- 9 API Endpoints Reference
- 10 Configuration & Environment Variables
- 11 Decision Codes & Auto-Response Mapping
- 12 RAG / Knowledge Base Architecture
- 13 Offline Tooling & Scripts

1. System Architecture Overview

This system is a **modular monolith** (not separate microservices). A single **FastAPI process** (Python 3) exposes all REST + SSE endpoints on **port 8020**. A **Next.js 16** frontend proxies every request to that FastAPI backend. The Python codebase is organised into independent **logical service packages** under `backend_modified/backend/`. All persistence is **file-backed JSON + CSV** with no external database — ChromaDB is the only optional external store (for RAG embeddings).

Layer	Technology	Entry Point	Port
Operator UI	Next.js 16 / React 18 / TypeScript	frontend/app/page.tsx	3000
BFF API Routes	Next.js API Routes (proxy to FastAPI)	frontend/app/api/*/route.ts	3000
Backend API	FastAPI 0.1x / Uvicorn	backend/fastapi_server.py	8020
LLM Layer	OpenAI SDK (gpt-4o, gpt-4o-mini)	openai.ChatCompletion / Embeddings	External
Vector Store	ChromaDB (persistent)	backend/rag/service.py	Local
Persistence	JSON files + CSV (file-backed)	backend/common/config.py paths	Disk
Email I/O	imaplib (IMAP4_SSL) + smtplib (SMTP)	backend/email_ingestion/service.py	993/587

2. Tech Stack

Backend (Python)

- **FastAPI** — REST + SSE web framework
- **Uvicorn** — ASGI server (host:0.0.0.0, port:8020)
- **Pydantic v2** — request/response validation, models
- **OpenAI Python SDK** — gpt-4o, gpt-4o-mini, embeddings
- **ChromaDB** — local persistent vector store for RAG
- **pypdf** — PDF text extraction (pure Python)
- **Pillow** — image processing for Vision API uploads
- **pdf2image** — PDF page → image for Vision pipeline
- **imaplib** (stdlib) — IMAP4_SSL inbox polling
- **smtplib** (stdlib) — SMTP email sending (TLS)
- **email** (stdlib) — MIME parsing, RFC822 decoding
- **json / csv** (stdlib) — file-backed persistence
- **threading** (stdlib) — concurrent sync with locks

External Services

- **OpenAI API** — Chat completions + Embeddings
- **Gmail IMAP** (imap.gmail.com:993) — inbox polling
- **Gmail SMTP** (smtp.gmail.com:587) — outbound email
- **SendGrid Inbound** — webhook route exists (optional)

Frontend (TypeScript / Node)

- **Next.js 16** — full-stack React framework (App Router)
- **React 18** — UI library
- **TypeScript** — type-safe codebase
- **Tailwind CSS** — utility-first styling
- **Framer Motion** — UI animations
- **Recharts** — dashboard charts
- **react-hook-form + zod** — form validation
- **jsPDF** — client-side PDF export of decision packs
- **LangChain / LangGraph** — legacy agent code (unused live)

Configuration / Tooling

- **.env** (repo root) — all secrets & toggles
- **requirements.txt** — Python deps
- **package.json** — Node deps + scripts
- **tsconfig.json / next.config.js / tailwind.config.js**
- **CORS** — configurable via CORS_ORIGINS env var
- **SSE (Server-Sent Events)** — streaming sync progress

3. Microservices / Logical Modules

All modules are Python packages under `backend_modified/backend/`. They communicate via direct Python imports — not network calls.

Email Ingestion

`email_ingestion/service.py`

IMAP4_SSL inbox polling. Classifies emails as FAQ, complaint, duplicate, or own-address. Sends ACK email. Runs auto-closure check after every sync. Uses a UID cache to avoid re-processing. Protected by a threading Lock to prevent concurrent sync races.

Tech: imaplib, smtplib, threading, OpenAI gpt-4o-mini (complaint classifier)

Ingested Complaints Store

`ingested_complaints/service.py`

JSON-backed store for raw inbound emails. Manages complaint lifecycle: save, dedup, ACK-slot claim, thread append, mark-processed, auto-closed, status update. Stores attachments on disk under `data/ingested-attachments/`.

Tech: json, threading (write lock), pathlib

Extraction Service

`extraction/service.py`

Uses OpenAI gpt-4o to extract 12+ structured fields from the email body. Processes PDF (pypdf), DOCX (zipfile XML), image (Vision API base64) attachments. Returns `extractedFields`, `documents` list, and `evidence` list with per-field confidence scores.

Tech: OpenAI gpt-4o (chat + vision), pypdf, zipfile, Pillow, base64

Validation Engine

`validation/service.py`

Runs 5 sequential validation checks: product match (F1 token-overlap scoring), customer–product ownership mapping, eligibility (physical damage / unauthorised repair / unsupported product), warranty (date arithmetic), and document presence. Reads `customers.json`, `complaints.json`, `product_service.json`.

Tech: json, re (regex patterns), datetime arithmetic

Decision Service

`decision/service.py`

Builds the complete decision pack. Calls customer grounding, derives auto-decision code (APPROVE_REPAIR / APPROVE_REPLACEMENT / REQUEST_DOCUMENTS / DESK_REJECT / INVESTIGATE / TROUBLESHOOTING_REQUIRED). Produces audit trail, evidence summary, processing metrics.

Tech: `decision/customer_grounding.py`, `decision/resolution_rules.py`

Customer Grounding

`decision/customer_grounding.py`

Looks up customer, complaint history, product, and action records from the 4 core JSON files. Scores confidence (0–1) and recommends AUTO_APPROVE / MANUAL_REVIEW / INVESTIGATE / ESCALATE. In-memory cache per process startup.

Tech: customers.json, complaints.json, complaint_action.json, product_service.json

RAG Service

[rag/service.py](#)

Semantic search over Samsung product manual knowledge base. Embeds query with text-embedding-3-small, queries ChromaDB with hard product_id filter. Falls back to JSON knowledge chunks, then FAQ.csv. Generates grounded answer with gpt-4o. Also used for FAQ email classification and product detection.

Tech: ChromaDB, OpenAI text-embedding-3-small, OpenAI gpt-4o, gpt-4o-mini, FAQ.csv

Auto Response Service

[auto_response/service.py](#)

Sends SMTP emails for all 7 decision codes. Strips markdown from LLM output. Sets In-Reply-To / References headers so replies land in the customer's email thread. Supports: REQUEST_DOCUMENTS, DESK_REJECT (5 sub-reasons), APPROVE_REPAIR, APPROVE_REPLACEMENT, TROUBLESHOOTING_REQUIRED, INVESTIGATE, AUTO_CLOSE.

Tech: smtplib, email.mime, re (markdown stripper)

Orchestrator

[process_complaint/orchestrator.py](#)

Coordinates the full 6-step pipeline: pre-check → extraction → validation → decision → RAG injection → auto-email → save. Entry point: process_complaint(ingested_id). Called by POST /api/process-complaint.

Tech: Imports all services above

FAQ Resolution

[faq_resolution/service.py](#)

Classifies an email as FAQ using keyword + LLM check. Delegates to rag/service.py for answer generation. Saves FAQ interactions to faq-emails.json.

Tech: rag/service.py, OpenAI gpt-4o-mini

Dashboard Service

[dashboard/service.py](#)

Manages processed complaint index (complaints-index.json) and per-complaint JSON files. Appends rows to complaints-history.csv. Computes KPI statistics: totals, by-decision, by-category, warranty breakdown, auto-email stats.

Tech: json, csv, datetime, pathlib

Appointments Service

[appointments/service.py](#)

CRUD operations on appointments.json. Books engineer visits linked to complaint IDs.

Tech: json, pathlib

Common / Config

[common/config.py](#) + [models.py](#)

Centralised path resolution (PROJECT_ROOT, DATA_DIR, all file paths). Pydantic v2 request/response models shared across FastAPI routes.

Tech: Pydantic v2, pathlib, os.getenv

4. LLM Models Used

Model	Default?	Where Used	Purpose	Tokens
gpt-4o	Yes*	extraction/service.py rag/service.py fastapi_server.py	Email field extraction, document classification, Vision image analysis, RAG answer generation, fallback troubleshooting (no RAG chunks)	1500
gpt-4o-mini	Classifier	email_ingestion/service.py faq_resolution/service.py rag/service.py	Complaint email classifier (yes/no), FAQ email classifier (FAQ/COMPLAINT), FAQ CSV matcher	5–10
text-embedding-3-small	Fixed	rag/service.py build_embeddings.py ingest_pdfs.py	Query embedding for semantic search, Chunk embedding during offline ingestion	N/A

* The active model for extraction and RAG answers is configurable via `OPENAI_MODEL` env var (defaults to `gpt-4o`). The complaint classifier in `email_ingestion` also reads `OPENAI_MODEL` but defaults to `gpt-4o-mini` when unset. The RAG answer model (`ANSWER_MODEL`) is hardcoded to `gpt-4o` in `rag/service.py`.

LLM Call Details by Module

Module / File	Model	Prompt Type	Max Tokens	Temperature
extraction/service.py (extract_from_email)	gpt-4o	System + User JSON schema	1500	0.1
extraction/service.py (extract_from_document)	gpt-4o	System + User doc classify	1500	0.1
extraction/service.py (analyze_image_evidence)	gpt-4o Vision	System + User + image_url	2500	0.1
email_ingestion/service.py (_classify_complaint_by_llm)	gpt-4o-mini	System + User yes/no	5	0
rag/service.py (_is_faq_query)	gpt-4o-mini	System + User FAQ/COMPLAINT	5	0
rag/service.py (_faq_csv_fallback)	gpt-4o-mini	System + User A-number match	10	0
rag/service.py (generate_answer)	gpt-4o	System (product expert) + context + question	1000	0.1
fastapi_server.py (/api/troubleshoot fallback)	gpt-4o	User prompt only (no RAG chunks)	600	0.4
rag/service.py (semantic_search)	text-embedding-3-small	Embedding input	—	—

5. Data Files & Schemas

Core Reference Data (backend_modified/data/)

File	Type	Purpose	Used By
customers.json	Array of objects	Customer master: id, name, email, phone, address, loyalty_tier, registered_products, product_purchase_dates	validation, decision/customer_grounding, rag/_verify_customer_product
complaints.json	Array of objects	Historical complaint records: complaint_id, customer_id, product_id, complaint_type, current_status, priority, is_escalated, sla_hours, refund_total	decision/customer_grounding, validation (ownership fallback)
complaint_action.json	Array of objects	Actions taken per complaint: action_id, complaint_id, action_description, action_date, agent_id	decision/customer_grounding
product_service.json	Array of objects	Product catalogue: product_id, product_name, brand_name, model_number, product_category, warranty_period_months, authorized_sellers	validation, decision/customer_grounding, rag/_verify_customer_product
FAQ.csv	CSV (Question, Answer)	Fallback Q&A pairs if RAG and ChromaDB find no chunks	rag/_faq_csv_fallback
ingested-complaints.json	Array of objects	Queue of raw inbound emails: id, from, subject, emailBody, attachments[], processingStatus, messageId, threadId	ingested-complaints/service (CRUD), process_complaint/orchestrator
imap-sync-cache.json	Dict of cache keys	Last IMAP UID per account+mailbox (and :faq suffix). Prevents re-fetching already-seen emails.	email_ingestion/service
faq-emails.json	Array of objects	Stored FAQ interactions: from, to, subject, body, matched_faq, timestamp	faq_resolution/service, fastapi_server
appointments.json	Array of objects	Engineer appointments: appointment_id, complaint_id, date, time, engineer_name, location, notes	appointments/service, fastapi_server
rag_knowledge/*.json	Array of chunk objects	Pre-extracted manual text chunks: text, product_id, category, page_num, page_range	rag/service (JSON fallback when ChromaDB unavailable)
processed-complaints/complaints-index.json	Array of objects	Fast lookup index for processed complaints: complaintId, warrantyStatus, autoDecision, decisionConfidence, filePath	dashboard/service
processed-complaints/complaints-history.csv	CSV (12 columns)	Append-only audit log of all processed complaints	dashboard/service (export)
processed-complaints/CMP-*.json	Full decision pack	Complete complaint record per processed complaint: extraInfo, autoDecision, decisionConfidence, filePath	decision/service, fastapi_server, auto-email
ingested-attachments/<ING-id>/filename	Binary files	Raw email attachments (PDFs, images, DOCX) for processing	email_ingestion/service
data/chroma_db/	ChromaDB persist dir	Vector embeddings for Samsung product manual chunks.	rag/service (persistent search)

6. End-to-End Workflow — Step by Step

Two entry paths: (A) **FAQ Path** — email is a general query; answered automatically via RAG/FAQ and stored. (B) **Complaint Path** — email is a genuine complaint; goes through full 6-step AI pipeline. Both paths start from the same IMAP sync.

6.1 Email Ingestion & Classification

STEP 0	IMAP Inbox Polling · email_ingestion/service.py
	Trigger: POST /api/sync-inbox (or /api/sync-inbox/stream for SSE) — manual or scheduled
	Connects to Gmail via IMAP4_SSL (imap.gmail.com:993)
	Uses UID cache (imap-sync-cache.json) → only fetches emails with UID > last seen
	First sync: fetches last 100 emails (configurable via IMAP_SYNC_MAX_EMAILS)
	Threading lock (_SYNC_LOCK) prevents two concurrent syncs from creating duplicates
	Self-email guard: skips emails FROM SENDER_EMAIL (our own auto-replies in All Mail)
	Dedup check: message-ID + subject from date hash against existing complaint & FAQ IDs
	Parses RFC822 message: extracts subject, from, to, body (text/plain preferred), attachments

STEP 0b	Complaint vs. FAQ Classification
	Step A: Keyword scan (_has_relevant_keywords) — 40+ regex patterns for complaint signals
	Step B: Strong keyword check (_has_strong_keywords) — unambiguous escalation phrases
	Step C: LLM filter — OpenAI gpt-4o-mini asked yes/no: 'Is this a genuine complaint?'
	→ If COMPLAINT_FILTER_ENABLED=false, skip LLM and ingest all keyword-matched emails
	Step D: FAQ check (runs BEFORE complaint filter) — process_faq_email() in rag/service.py
	→ If _is_faq_query() returns True → FAQ branch (section 6.2)
	→ If complaint signals detected → Complaint branch (section 6.3)
	→ Neither → skippedNoComplaint counter incremented
	UID checkpointed after EVERY email (prevents re-firing auto-replies on restart)

6.2 FAQ Branch · rag/service.py + faq_resolution/service.py

STEP FAQ	FAQ Detection & Auto-Answer · rag/service.py
----------	--

Entry: process_faq_email(from, to, subject, body, message_id)
1. _is_faq_query(): keyword exclusion (complaint terms = not FAQ) + inclusion (how/what/can I)
then LLM confirm: gpt-4o-mini → 'FAQ' or 'COMPLAINT'
2. Product detection: match_product_from_text() — keyword/alias dict → product_id
Supports 8 Samsung products (Galaxy S26, A15, Neo QLED, OLED, Fridges, ACs)
3. Customer verification: _verify_customer_product() — check CRM, registered_products, warranty
Verification failures → _send_verification_failed_email() → 4 custom rejection templates
4. Semantic search: semantic_search(query, top_k=5, product_id=X) → ChromaDB with filter
Fallback: JSON knowledge chunks if ChromaDB empty
5. Answer generation: generate_answer() → gpt-4o with product-expert system prompt
6. CSV fallback: _faq_csv_fallback() → gpt-4o-mini matches question to FAQ.csv rows
7. Generic fallback: 'We will respond within 2 business days' if all above fail
8. _send_faq_email() → SMTP reply personalised with customer name + verification status
9. save_faq_email() → append to faq-emails.json
Result stored: faq-emails.json · Dedup key added to prevent re-processing

6.3 Complaint Branch — AI Processing Pipeline

Once an email passes the complaint filter, it is saved as an **Ingested Complaint** (ING-* ID) in ingested-complaints.json. An ACK email is immediately sent to the customer. The operator then triggers processing from the UI or API, which runs the full pipeline below. The orchestrator entry point is process_complaint/orchestrator.py → process_complaint(id).

STEP
1

Save Ingested Complaint · ingested_complaints/service.py

save_ingested_complaint() called with: from, to, subject, body, attachments, source='imap'

Generates ING-- ID, sets processingStatus='pending'

Saves email body + metadata to ingested-complaints.json (append under write lock)

Saves raw attachment bytes to data/ingested-attachments//

ACK email sent immediately: 'We have received your complaint, ref ING-xxx'

→ Uses claim_ack_slot() (atomic flag) to prevent duplicate ACK on restart

→ ACK recorded in thread via add_email_to_thread() for UI display

Auto-closure check runs after every sync (ENABLE_AUTO_CLOSURE=true to enable)

→ Complaints idle for AUTO_CLOSURE_DAYS (default 7) get auto-closure email

STEP
0

Pre-Check: Customer Guard · orchestrator.py

Before any LLM call, checks if complaintRef (CUST#####) exists in CRM

Uses find_customer() from decision/customer_grounding.py

If customer NOT found → immediately creates DESK_REJECT decision pack

→ Sends DESK_REJECT auto-email (customer_not_found reason)

→ Saves to dashboard + marks ingested complaint processed

→ Returns without running extraction, validation, or decision pipeline

This saves OpenAI API costs for completely unknown accounts

STEP
2

AI Extraction · extraction/service.py

Entry: <code>extract_claim_information(claim_id, email_body, attachments)</code>
Email extraction: <code>extract_from_email()</code> → gpt-4o with JSON schema prompt
Extracts: <code>customerName</code> , <code>customerEmail</code> , <code>customerPhone</code> , <code>complaintRef</code> ,
<code>complaintDate</code> , <code>purchaseDate</code> , <code>complaintType</code> (11 electronics categories),
<code>productOrService</code> , <code>productModel</code> , <code>serialNumber</code> , <code>description</code> , <code>desiredResolution</code>
Each field includes a confidence score (0.0–1.0)
Attachment processing:
• Image files (.jpg/.png/.gif/.webp): Vision API → <code>analyze_image_evidence()</code>
Returns: <code>defect_type</code> , <code>severity</code> , <code>evidence_quality</code> , <code>supports_complaint</code>
• PDF files: pypdf text extraction → <code>extract_from_document()</code>
• DOCX files: zipfile XML parse → <code>extract_from_document()</code>
• Scanned/binary PDFs: noted as '[no text layer found]'
Document classification: Invoice / Receipt / Screenshot / PhotoEvidence / Other
Returns: { <code>extractedFields</code> , <code>documents[]</code> , <code>evidence[]</code> , <code>errors[]</code> }

STEP
3

Validation Engine · validation/service.py

Entry: run_validation(extracted_fields, documents)

Loads: customers.json, complaints.json, product_service.json

Check 1 — Product Validation (validate_product):

Token-overlap F1 scoring between extracted product name and catalogue

Exact match → 1.0; model substring → 0.9+; F1 token overlap; min threshold 0.45

Supported categories: smartphone, laptop, tablet, camera, earbuds, smartwatch

Result: productCategory, matchedProductId, warrantyMonths

Check 2 — Customer-Product Mapping (validate_customer_product_mapping):

Resolves customer by: complaintRef (CUST#) → email → full name

Checks product in customer.registered_products[]

Ownership fallback: prior complaint history in complaints.json

Failure → autoDecision = DESK_REJECT (product_not_registered)

Check 3 — Eligibility (validate_eligibility):

Scans description for physical damage patterns (dropped, water damage, smashed)

Scans for unauthorised repair (third-party repair, warranty void, tampered)

Scans for unsupported product types (furniture, appliances, food)

Any match → autoDecision = DESK_REJECT

Check 4 — Warranty Validation (validate_warranty):

Extracts purchase date from: document keyFields → email body

Warranty = matched_product.warranty_period_months (default 12)

Calculates expiry = purchase_date + months × 30.44

OUT_OF_WARRANTY → NOT a desk reject; proceeds as paid-service case

UNKNOWN date → passes with benefit of doubt

Check 5 — Document Validation (validate_documents):

Required: purchase invoice (hard fail if missing → REQUEST_DOCUMENTS)

Soft checks: product model, serial number

Priority: DESK_REJECT > REQUEST_DOCUMENTS > None

Output: { autoDecision, allChecksPassed, warrantyStatus, productCategory,

matchedProduct, validationResults[] }

STEP
4

Decision Engine · decision/service.py

Entry: build_decision_pack(ingested_id, email, extraction, validation, ...)
Customer Grounding (decision/customer_grounding.py):
1. find_customer() by complaintRef / email / name
2. get_customer_complaints() → filter to current product_id
3. get_complaint_actions() for each complaint
4. get_product() for product details
Confidence score formula: 0.30 (customer) + 0.25 (history) + 0.25 (product) + 0.20 (base)
Recommendation thresholds: ≥0.85 → AUTO_APPROVE; ≥0.65 → MANUAL_REVIEW; else INVESTIGATE
Escalated or CRITICAL/URGENT priority → ESCALATE (→ INVESTIGATE decision)
_derive_auto_decision() logic:
1. DESK_REJECT from validation? → map reject_reason to specific code
2. REQUEST_DOCUMENTS + troubleshooting signals (software/settings keywords)? → TROUBLESHOOTING_REQUIRED (bypass invoice requirement)
3. REQUEST_DOCUMENTS (no troubleshooting) → REQUEST_DOCUMENTS
4. Grounding = CUSTOMER_NOT_FOUND → DESK_REJECT
5. Grounding = ESCALATE → INVESTIGATE
6. AUTO_APPROVE + wants_replacement → APPROVE_REPLACEMENT
7. AUTO_APPROVE + needs_troubleshooting → TROUBLESHOOTING_REQUIRED
8. AUTO_APPROVE → APPROVE_REPAIR
9. Default → INVESTIGATE
Output: full decision pack with audit trail, evidence summary, customer info

STEP
5

RAG Injection (Conditional) · orchestrator.py → rag/service.py

Triggered when: autoDecision ∈ { TROUBLESHOOTING_REQUIRED, INVESTIGATE }

Skipped for: DESK_REJECT, REQUEST_DOCUMENTS, APPROVE_REPAIR, APPROVE_REPLACEMENT

1. match_product_from_text(subject, description) → product_id via keyword dict

(exact_name match > keyword/alias match; 8 Samsung products supported)

2. semantic_search(query, top_k=4, product_id=X)

→ Embeds query with text-embedding-3-small

→ Queries ChromaDB with hard product_id filter (no cross-product bleed)

→ Fallback: cosine similarity over JSON knowledge chunks

→ Returns top-4 chunks with text, product_name, category, page_range

3. generate_answer(question, chunks, product_id, product_name)

→ gpt-4o with Samsung Senior Technical Support Specialist persona

→ System rules: use only manual excerpts, numbered steps, no citations

→ Returns answer string + sources list

4. auto_decision overridden to REQUEST_DOCUMENTS (for email template routing)

rag_troubleshooting_steps stored → passed to send_auto_response()

If RAG fails (exception): auto_decision still set to REQUEST_DOCUMENTS

If no chunks found: /api/troubleshoot falls back to direct gpt-4o general advice

STEP
6

Auto Email Response · auto_response/service.py

Entry: `send_auto_response(to, customer_name, complaint_id, decision, ...)`

Email routing by decision code:

- `troubleshooting_steps` present → TROUBLESHOOTING email (takes priority)
- `REQUEST_DOCUMENTS` → list missing docs, ask for reply with attachments
- `DESK_REJECT` + `physical_damage` → physical damage template
- `DESK_REJECT` + `unauthorized_repair` → void warranty template
- `DESK_REJECT` + `unsupported_product` → unsupported category template
- `DESK_REJECT` + `product_not_registered` → ownership mismatch template
- `DESK_REJECT` + `customer_not_found` → account not found template
- `APPROVE_REPAIR` / `APPROVE_REPLACEMENT` → approval with next-steps
(out-of-warranty adds paid-service notice)
- `AUTO_CLOSE` → inactivity closure (N days configurable)
- `INVESTIGATE` → manual review acknowledgement

`_strip_markdown()`: removes **bold**, *italic*, ### headings, - bullets from LLM output

SMTP: gmail.com:587 with STARTTLS → sends plain + HTML MIME multipart

Thread headers: In-Reply-To + References set so email lands in customer's thread

Result appended to complaint thread via `add_email_to_thread()`

STEP
7

Save to Dashboard · `dashboard/service.py`

Entry: `save_processed_complaint(complaint_data)`

Writes full decision pack to `processed-complaints/CMP-.json`

Updates `complaints-index.json` (fast lookup index)

Appends row to `complaints-history.csv` (first write adds header)

`mark_ingested_complaint_processed()` sets `processingStatus='processed'` in `ingested-complaints.json`

Processing metrics recorded:

`totalProcessingTime`, `extractionTime`, `validationTime`, `averageHandleTime`

Dashboard KPIs (GET `/api/dashboard/kpis`):

`totalComplaints`, `complaintsThisWeek/Month`, `complaintsByType`

`complaintsByDecision`, `warrantyStatusCounts`, `complaintsByCategory`

`autoEmailsSent`, `autoEmailsAttempted`, `avgResolutionConfidence`

`complaintsByDate` (last 14 days), `recentComplaints` (last 5)

7. Cross-Reference Map

Which module imports / calls which other module or data file:

Module	Imports / Reads	Writes / Updates
fastapi_server.py	All service modules, common/config, common/models	HTTP responses only
process_complaint/ orchestrator.py	extraction/service validation/service decision/service decision/customer_grounding ingested_complaints/service auto_response/service dashboard/service rag/service (conditional)	Triggers writes in all subservices below
email_ingestion/ service.py	faq_resolution/service ingested_complaints/service auto_response/service common/config	ingested-complaints.json imap-sync-cache.json faq-emails.json (via faq_resolution)
extraction/service.py	OpenAI API (gpt-4o + vision) common/config	None (returns dict)
validation/service.py	customers.json complaints.json (ownership fallback) product_service.json	None (returns dict)
decision/service.py	decision/customer_grounding decision/resolution_rules	None (returns dict)
decision/ customer_grounding.py	customers.json complaints.json complaint_action.json product_service.json	None (in-memory cache)
rag/service.py	ChromaDB (chroma_db/ rag_knowledge/*.json FAQ.csv customers.json product_service.json OpenAI (embeddings + gpt-4o + gpt-4o-mini)	Sends SMTP email (FAQ reply)
faq_resolution/service.py	rag/service.py common/config	faq-emails.json
auto_response/service.py	common/config (ENV_FILE for SMTP creds)	Sends SMTP email
dashboard/service.py	common/config (file paths)	CMP-*.json complaints-index.json complaints-history.csv
ingested_complaints/ service.py	common/config	ingested-complaints.json ingested-attachments/**
appointments/service.py	common/config	appointments.json
ingest_pdfs.py build_embeddings.py	PDF files (manual PDFs) OpenAI (gpt-4o vision + embeddings)	rag_knowledge/*.json chroma_db/ (ChromaDB)

8. Frontend (Next.js) Layer

8.1 UI Pages · frontend/app/

File	Page / Role
app/page.tsx	Main app shell — staged navigation: Home → Review → Decision → Dashboard
app/login/page.tsx	Login screen (auth guard)
app/layout.tsx	Root layout with Header
components/HomePage.tsx	Landing / empty-state view
components/ReviewPage.tsx	Ingested complaints list + trigger processing
components/DecisionPage.tsx	Decision pack viewer — evidence, validation, grounding, auto-email
components/DashboardPage.tsx	KPI charts (Recharts), complaint list, CSV export, troubleshoot panel
components/FAQPage.tsx	FAQ email history viewer
components/MailMetaPanel.tsx	Email thread viewer (inbound + outbound)
components/ConfigModal.tsx	API URL configuration modal
components/ClaimSummaryBar.tsx	Compact complaint info bar
components/Header.tsx	Top navigation bar

8.2 API Routes (BFF / Proxy Layer) · frontend/app/api/

Route	Method	Proxies To FastAPI	Purpose
api/process-claim	POST	POST /api/process-complaint	Trigger AI processing pipeline
api/claims	GET/POST	GET /api/complaints	List / save processed complaints
api/claims/[id]	GET/PATCH	GET/PATCH /api/complaints/:id	Get / update complaint
api/claims/export	GET	GET /api/complaints/export/csv	CSV export
api/ingested-claims	GET	GET /api/ingested-complaints	List ingested
api/ingested-claims/[id]	GET/DELETE	GET /api/ingested-complaints/:id	Single ingested complaint
api/ingested-claims/[id]/thread	GET/POST	GET/POST /api/ingested-complaints/:id/thread	Email thread
api/ingested-claims/[id]/attachments	GET	GET /api/ingested-complaints/:id	Attachment download
api/ingested-claims/clear	POST	POST /api/ingested-complaints/clear	Clear all ingested + reset UID cache
api/sync-inbox	POST	POST /api/sync-inbox	Trigger IMAP sync
api/sync-inbox/stream	GET	GET /api/sync-inbox/stream	SSE streaming sync progress
api/sync-faq	POST	POST /api/sync-faq	FAQ-only IMAP sync
api/sync-faq/stream	GET	GET /api/sync-faq/stream	SSE FAQ sync progress
api/faq	GET	GET /api/faq	FAQ Q&A list (FAQ.csv)
api/faq/emails	GET	GET /api/faq/emails	FAQ email history
api/faq/emails/clear	POST	POST /api/faq/emails/clear	Clear FAQ history + UID cache
api/dashboard/kpis	GET	GET /api/dashboard/kpis	KPI statistics
api/troubleshoot	POST	POST /api/troubleshoot	RAG troubleshoot (dashboard)

api/appointments	GET/POST	GET/POST /api/appointments	Engineer appointments
api/decision-pack/pdf	POST	Client-side (jsPDF)	Generate PDF of decision pack
api/send-email	POST	Direct SMTP (internal)	Manual email send
api/drafts	GET/POST	Local or FastAPI	Draft management
api/auth/login	POST	Local auth check	Session login
api/complaints/[id]/status	PATCH	PATCH /api/complaints/:id/status	Update complaint status
api/webhooks/sendgrid-inbound	POST	Parses SendGrid payload	Inbound email via SendGrid

8.3 Library Files · frontend/lib/ + frontend/types/

File	Purpose
lib/api-config.ts	getApiUrl() — reads NEXT_PUBLIC_API_URL, defaults to http://localhost:8020
lib/backend.ts	Typed API client functions for all FastAPI endpoints
lib/normalizeClaim.ts	Normalises FastAPI response to internal Claim type
lib/clientCache.ts	Client-side in-memory cache for claims list
lib/auth/authGuard.ts	Session-based route protection
lib/agents/orchestrator.ts	LangGraph agent orchestrator (legacy/unused in live flow)
lib/services/openai.ts	OpenAI client for frontend (legacy, defaults gpt-4-1106-preview)
lib/claimProcessor.ts	Frontend claim processing (NOT imported live — backend does processing)
types/claims.ts	TypeScript types: Claim, DecisionPack, ValidationResult, etc.
hooks/useSync.ts	SSE hook for streaming sync progress to UI

9. API Endpoints Reference · fastapi_server.py

Method	Path	Handler	Purpose
GET	/health	health_check()	Service health + version
GET	/	root()	API info + docs link
POST	/api/process-complaint	process_complaint_endpoint()	Full pipeline trigger
GET	/api/ingested-complaints	list_ingested_complaints(full=bool)	List ingested (refs or full)
GET	/api/ingested-complaints/{id}	get_ingested_complaint_endpoint()	Single ingested complaint
GET	/api/ingested-complaints/{id}/thread	get_ingested_complaint_thread()	Email thread
POST	/api/ingested-complaints/{id}/thread	add_to_ingested_complaint_thread()	Append outbound email
POST	/api/ingested-complaints/clear	clear_ingested_complaints_endpoint()	Delete all + reset UID
GET	/api/complaints	list_processed_complaints(includeDeskRejected=bool)	Processed list
POST	/api/complaints	save_complaint_endpoint()	Save decision pack
GET	/api/complaints/export/csv	export_complaints_csv()	CSV download
GET	/api/complaints/{id}	get_complaint_endpoint()	Single processed complaint
PATCH	/api/complaints/{id}/status	update_complaint_status_endpoint()	Status update
GET	/api/dashboard/kpis	get_dashboard_kpis_endpoint()	KPI statistics
POST	/api/sync-inbox	sync_inbox_endpoint()	IMAP sync (blocking)
GET	/api/sync-inbox/stream	sync_inbox_stream_endpoint()	SSE sync progress
POST	/api/appointments	book_appointment_endpoint()	Book engineer visit
GET	/api/appointments	list_appointments_endpoint(complaintId=str)	List appointments
GET	/api/appointments/{id}	get_appointment_endpoint()	Single appointment
GET	/api/faq	list_faqs()	FAQ Q&A list from FAQ.csv
GET	/api/faq/emails	get_faq_related_emails()	FAQ email history
POST	/api/sync-faq	sync_faq_endpoint()	FAQ-only IMAP sync
GET	/api/sync-faq/stream	sync_faq_stream_endpoint()	SSE FAQ sync progress
POST	/api/faq/emails/clear	clear_faq_emails_endpoint()	Clear FAQ history + UID
POST	/api/troubleshoot	get_troubleshooting_steps()	RAG troubleshoot (dashboard)

10. Configuration & Environment Variables · .env

Variable	Default	Used By	Purpose
OPENAI_API_KEY	required	All LLM calls	OpenAI authentication key
OPENAI_MODEL	gpt-4o	extraction, troubleshoot	Override extraction/RAG model
SENDER_EMAIL	required	email_ingestion, auto_response, rag	Gmail address for IMAP + SMTP
EMAIL_PASSWORD	required	email_ingestion, auto_response, rag	Gmail App Password (spaces stripped)
IMAP_HOST	imap.gmail.com	email_ingestion	IMAP server hostname
IMAP_PORT	993	email_ingestion	IMAP SSL port
IMAP_MAILBOX	INBOX	email_ingestion	Gmail tries All Mail first
IMAP_SYNC_INCLUDE_READ	true	email_ingestion	Sync already-read emails
IMAP_SYNC_MAX_EMAILS	100	email_ingestion	Max emails per first sync
IMAP_SSL_VERIFY	false	email_ingestion	SSL certificate verification
SMTP_HOST	smtp.gmail.com	auto_response, rag	SMTP server hostname
SMTP_PORT	587	auto_response, rag	SMTP TLS port
COMPLAINT_FILTER_ENABLED	true	email_ingestion	Enable LLM complaint classifier
ENABLE_AUTO_CLOSURE	false	email_ingestion	Auto-close idle complaints
AUTO_CLOSURE_DAYS	7	email_ingestion	Days of inactivity before closure
FAQ_CSV_FILE	data/FAQ.csv	rag	Override FAQ CSV path
API_HOST	0.0.0.0	fastapi_server	FastAPI bind address
API_PORT	8020	fastapi_server	FastAPI bind port
API_RELOAD	false	fastapi_server	Uvicorn auto-reload
CORS_ORIGINS	*	fastapi_server	Allowed CORS origins (comma-separated)
PROJECT_ROOT	auto	common/config	Override project root path
DATA_DIR	PROJECT_ROOT/data	common/config	Override data directory
NEXT_PUBLIC_API_URL	http://localhost:8020	frontend/lib/api-config.ts	FastAPI base URL for frontend

11. Decision Codes & Auto-Response Mapping

Code	Trigger Condition	Email Template	Confidence
APPROVE_REPAIR	Customer & product verified (AUTO_APPROVE_grounding) No replacement signal. Within warranty → free; Out of warranty, 48h contact	_build_approval_body() Arranges repair	0.85
APPROVE_REPLACEMENT	AUTO_APPROVE grounding + replacement signal (replace, DOA, screen crack, water damage)	_build_approval_body() Arranges replacement	0.88
REQUEST_DOCUMENTS	Validation: purchase invoice missing. Not a software/settings issue.	_build_request_documents_body() Lists missing docs	0.90
TROUBLESHOOTING_REQUIRES_INVESTIGATION	Software/settings signals detected OR RAG injection triggered (TROUBLESHOOTING_MESSAGE) Overrides REQUEST_DOCUMENTS if software.	_build_troubleshooting_body() Provides RAG generated steps	0.90
INVESTIGATE	Insufficient verified data (grounding score < 0.65) Or escalated complaint.	_build_investigate_body() 2-5 business day review	0.60–0.75
DESK_REJECT (customer_not_found)	complaintRef not in customers.json. Pre-check guard OR customer_product_mapping	_build_customer_not_found_body() Ask to verify account	1.00
DESK_REJECT (product_not_registered)	Customer exists but product not in customer.registered_products[] and no complaint	_build_desk_reject_body() Ownership mismatch template	0.97
DESK_REJECT (physical_damage)	Description contains: dropped, water damage, smashed, cracked+impact, accidental.	_build_desk_reject_body() Physical damage template	0.90
DESK_REJECT (unauthorized_repair)	Description contains: third-party repair, warranty void, tampered, self-repair.	_build_desk_reject_body() Void warranty template	0.92
DESK_REJECT (unsupported_product)	Product contains: furniture, clothing, food, washing machine, refrigerator, oven, etc.	_build_desk_reject_body() Unsupported category	0.95
AUTO_CLOSE	ENABLE_AUTO_CLOSURE=true AND complaint idle for AUTO_CLOSURE_DAYS days.	_build_auto_closure_body() Inactivity closure	N/A

12. RAG / Knowledge Base Architecture

12.1 Products Covered (PRODUCT_ID_TO_NAME + PRODUCT_KEYWORDS)

Product ID	Product Name	Keywords / Aliases
PROD-SM-S938	Samsung Galaxy S26 Ultra	s26 ultra, galaxy s26, sm-s938, s26ultra
PROD-SM-A155	Samsung Galaxy A15	galaxy a15, a15, sm-a155, galaxya15
PROD-QN90D-65	Samsung Neo QLED 4K QN90D	qn90d, neo qled, qn90, neo-qled
PROD-S85D-55	Samsung OLED S85D	oled s85d, s85d, samsung oled, s85
PROD-RF350-BD	Samsung Bespoke AI Double Door 350L	double door 350l, 350l, bespoke 350, rt39, rf350
PROD-RF653-SBS	Samsung Bespoke Convertible 5in1 653L Side by Side	side by side, rs76, bespoke 653, rf653, 5in1
PROD-AC-WFAI	Samsung Bespoke AI WindFree AC	windfree bespoke, bespoke ai, ar18cx, windfree ai
PROD-AC-WF15	Samsung WindFree 1.5 Ton Inverter AC	windfree 1.5, windfree ac, ar18by, wind free, wf15

12.2 RAG Pipeline

Offline Ingestion (run once)

ingest_pdfs.py: load Samsung product PDFs from configured directory

Text pages: pypdf text extraction

Image pages (scanned): pdf2image → Pillow → gpt-4o Vision API → text description

Chunk text: split into overlapping chunks with product_id + page metadata

Embed: text-embedding-3-small via OpenAI Embeddings API

Store: ChromaDB PersistentClient → collection 'samsung_manuals' with cosine metric

Save JSON: rag_knowledge/.json (fallback if ChromaDB unavailable)

Runtime: semantic_search()

Embed query with text-embedding-3-small

If ChromaDB available and has chunks:

```
→ collection.query(query_embeddings=[...], n_results=top_k, where={'product_id': {'$eq': X}})
```

```
→ Returns documents, metadatas (product_name, category, page_range), distances
```

Fallback: load JSON chunks → compute cosine similarity → sort → top_k

Hard product_id filter prevents cross-product answers (no S26 answer for A15 query)

Runtime: generate_answer()

Build context: join top-k chunks with [SOURCE: ProductName Manual — Pages X] prefix

System prompt: Samsung Senior Technical Support Specialist for

Rules: use only manual; numbered steps; no citations; speak as expert; no 'based on manual'

gpt-4o call: max_tokens=1000, temperature=0.1

Returns: answer string, sources[], confidence (1 - min_distance)

Fallback chain

1. ChromaDB with product_id filter → chunks found → generate_answer()
2. JSON knowledge files with product_id filter → chunks found → generate_answer()
3. FAQ.csv → gpt-4o-mini matches question to CSV rows → return answer
4. Generic fallback → 'Will respond within 2 business days'

13. Offline Tooling & Scripts

Script	Location	Purpose	When to Run
ingest_pdfs.py	backend_modified/	Primary ingestion: PDF → text (pypdf) or Vision (gpt-4o) → chunk → embed (text-embedding-3-small) → ChromaDB. Processes all PDFs in configured source directory.	Once per product manual update. Creates/updates chroma_db/ and rag_knowledge/.
build_embeddings.py	backend_modified/	Re-embeds existing rag_knowledge/*.json chunks into ChromaDB without re-doing PDF extraction. Faster than full re-ingestion when chunks already exist.	When ChromaDB is lost or needs rebuilding. Reads from existing JSON knowledge files.
ingest_pdf.py	backend_modified/	Single-file ingestion variant of ingest_pdfs.py. Ingest one specific PDF file by path.	Ad-hoc manual addition of a single product manual.
test_faq.py	backend_modified/	Seeds test FAQ emails into the system. Useful for testing FAQ detection and auto-response flow.	Development / QA testing only.
python -m backend.extract_complaint	backend_modified/	CLI: run AI extraction for a specific ingested complaint ID. Reads from ingested-complaints.json, prints JSON result.	Debugging extraction output for a specific complaint.
python -m backend.process_complaint	backend_modified/	CLI: run full pipeline for a specific ingested complaint. Same as POST /api/process-complaint but from terminal.	Debugging or manual processing without the API.
python -m backend.email_sync	backend_modified/	CLI: trigger one IMAP sync pass from terminal. Prints JSON result.	Testing IMAP connectivity and sync logic.
python -m backend.dashboard_kpi	backend_modified/	CLI: print dashboard KPIs from processed complaints. Reads complaints-index.json.	Quick KPI check from terminal.

System Summary

This is a Samsung consumer-electronics complaint management system. It ingests customer emails via IMAP, automatically classifies them as FAQs or complaints, and for complaints runs a 6-step AI pipeline: extraction (gpt-4o) → validation (rule-based) → decision (grounding + ML routing) → RAG troubleshooting (ChromaDB + gpt-4o) → auto-email (SMTP) → dashboard save. The entire backend is a single FastAPI process. The frontend is Next.js acting as a BFF proxy and operator UI. All persistence is file-backed JSON/CSV except ChromaDB for vector embeddings. Three OpenAI models are used: **gpt-4o** (extraction, RAG answers), **gpt-4o-mini** (classifiers), **text-embedding-3-small** (embeddings).